

Exercise 1 — Basic Maps and Graphs in R

BSTA 374 · 12 points · Due Thursday, September 10, 2026, 11:59 pm

Eric M. Delmelle · Lehigh University

2026-08-31

What this exercise is

Your first graded work in this course. It covers the first two weeks: reading spatial data with `sf`, understanding what a coordinate reference system is and moving between them, and producing honest choropleth and point maps with both `ggplot2` and `tmap`.

No data to download. Everything runs on the North Carolina SIDS data that ships inside the `sf` package, so your analysis will reproduce on my machine exactly as it does on yours — which is the point.

Each part ends with **starting code**. Copy it, fill in the blanks marked `_____`, and adapt it. It is scaffolding, not a solution: the mechanical lines are given so that your time goes into the parts that require judgement.

! Submission

Submit **one file** to Course Site:

- `LASTNAME_FI_Exercise1.qmd` — the source

Example: `DELMELLE_EM_Exercise1.qmd`. Put your name in the document header.

Due Thursday, September 10, 2026 at 11:59 pm. Late work loses 20% per day unless you arrange something with me in advance.

! Reproducibility is graded

Before you submit: restart R (`Ctrl/Cmd + Shift + F10`) and render the document from a clean session. If it does not run start to finish, it is incomplete — regardless of how good the maps are. This is the habit that matters most, and we start enforcing it now.

```
library(sf)
library(dplyr)
library(ggplot2)
library(tmap)
# add anything else you use -- and load it here, not halfway down
```

```
nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)
```

Part A — Read and describe the data (2 points)

Answer each in **one or two sentences of prose**, outside the code chunks, and show the code you used to find out.

1. How many features does `nc` contain, and what does one row represent?
2. What geometry type is stored? Why is it that type rather than **POLYGON**? (*Look at the coast.*)
3. What coordinate reference system is the data in? Give its name, say whether it is geographic or projected, and say how you found out.
4. In your own words, what do **BIR74** and **SID74** measure? Why does having both matter more than having either one alone?

Starting code

```
# A1 -- how big is it, and what are the columns?
nrow(nc)
names(nc)
nc                # the header block above the table says a lot

# A2 -- one geometry type per feature; count them
table(st_geometry_type(nc))

# A3 -- three ways of asking about the CRS
st_crs(nc)        # the full definition
st_crs(nc)$input  # the short name
st_is_longlat(nc) # TRUE = geographic (degrees), FALSE = projected (metres)

# A4 -- the help page documents every column
?nc
```

Part B — Coordinate reference systems (2 points)

1. Transform `nc` to **EPSG:32119** (NAD83 / North Carolina State Plane, in metres) and store it as a **new object**. Do not overwrite `nc` — you need both.
2. Plot the counties **before and after**, near enough to each other that a reader can compare them. One or two sentences on what visibly changed.

3. Compute the area of every county in **square kilometres**, using the projected version. Report the five largest.
4. Now compute areas from the *unprojected* object as well. Compare the two sets of numbers and explain, in two or three sentences, what **sf** did and why the projected version is the one you would report in a paper.

i Note

Step 4 will not give you an error, and the two sets of numbers will be close but not identical. That is the interesting part — the question is which *surface* each answer refers to.

Starting code

```
# B1
nc_proj <- st_transform(nc, 32119)

# B2 -- one plot each
p1 <- ggplot(nc) + geom_sf(fill = "grey90", colour = "white") + theme_void()
p2 <- ggplot(nc_proj) + geom_sf(fill = "grey90", colour = "white") + theme_void()

p1
p2
# side by side, if you like: library(patchwork); p1 + p2

# B3 -- st_area() returns square METRES, as a units object
nc_proj <- nc_proj |>
  mutate(area_km2 = as.numeric(units::set_units(st_area(nc_proj), km^2)))

nc_proj |>
  st_drop_geometry() |>
  slice_max(area_km2, n = 5) |>
  select(NAME, area_km2)

# B4 -- now do the same starting from nc (degrees) and compare
-----
```

Part C — Choropleth maps (5 points)

This is the largest part of the exercise, and it is as much about judgement as about code.

1. Add a column giving the **SIDS rate per 1,000 live births** for 1974–78.

2. Make a choropleth of the **raw counts** (SID74) and one of the **rate**, using `ggplot2`. Then write three or four sentences explaining why the two maps disagree, and which one you would put in front of a health department, and why.
3. Rebuild the rate map using **tmap**. Same information, other package.
4. Vary the **class intervals** on the rate map: produce it at least three ways — for instance equal intervals, quantiles, and Jenks natural breaks. Two or three sentences on how much the apparent geography of risk changes. Name the classification you would defend, and say why.
5. Every map you submit must have a **legend with units**, a **title**, thin boundaries that do not overwhelm the fill, and a **perceptually uniform, colour-blind-safe palette**. State in one sentence which palette you chose and why, including whether dark means high or low.

💡 What earns full marks here

Not the prettiest map — the map whose choices you can defend. A student who picks quantiles and explains the trade-off in two clear sentences scores higher than one who picks Jenks because it looked nicer.

Starting code

```
# C1 -- deaths per 1,000 live births
nc <- nc |>
  mutate(rate74 = _____)

# C2 -- counts, then rates. Same code, one variable changed.
ggplot(nc) +
  geom_sf(aes(fill = SID74), colour = "white", linewidth = 0.15) +
  scale_fill_viridis_c(option = "magma", direction = -1, name = "_____") +
  labs(title = "_____") +
  theme_void()

# other palettes to try:
#   scale_fill_viridis_c(option = "viridis" / "plasma" / "cividis" / "mako")
#   scale_fill_distiller(palette = "YlOrRd", direction = 1)
#   RColorBrewer::display.brewer.all() # shows every ColorBrewer option

# C3 -- the same map in tmap
tmap_mode("plot") # "view" makes it interactive

tm_shape(nc) +
  tm_polygons(fill = "rate74") +
  tm_title("_____")

# C4 -- change ONLY the classification, so the comparison is fair
tm_shape(nc) +
  tm_polygons(fill = "rate74",
    fill.scale = tm_scale_intervals(style = "quantile", n = 5)) +
```

```
tm_title("Quantiles")

# then repeat with style = "equal" and style = "jenks"
# other options: "pretty", "fisher"
```

⚠ A note on tmap versions

`tmap` changed its argument names in version 4. If an example you find online uses `tm_polygons(col = ...)` or `style = ...` directly and you get errors, check `packageVersion("tmap")` and read the help page rather than guessing. The code above is written for version 4.

Part D — A point layer (2 points)

1. Look up the approximate longitude and latitude of **four cities in North Carolina** and build a small table holding the name and the two coordinates. Cite where you got the coordinates.
2. Turn that table into an `sf` object with the correct CRS. State which EPSG code you used and why.
3. Plot the four cities **on top of** your rate choropleth, labelled legibly.
4. One sentence: what would go wrong if you built the `sf` object with the coordinate columns in the other order?

Starting code

```
# D1 -- North Carolina longitudes are NEGATIVE (west of Greenwich)
cities <- tibble(
  name = c("_____", "_____", "_____", "_____"),
  lon  = c(_____, _____, _____, _____),
  lat  = c(_____, _____, _____, _____)
)

# D2 -- coords takes the column names in x, y order: longitude FIRST
cities_sf <- cities |>
  st_as_sf(coords = c("lon", "lat"), crs = _____)

# D3 -- layers draw in order, so points come after polygons
ggplot() +
  geom_sf(data = nc, aes(fill = rate74), colour = "white", linewidth = 0.15) +
  geom_sf(data = cities_sf, size = 2.5, colour = "black") +
  geom_sf_text(data = cities_sf, aes(label = name), size = 3, nudge_y = 0.15) +
  scale_fill_viridis_c(option = "magma", direction = -1, name = "per 1,000") +
  theme_void()
```

```
# check your cities landed in North Carolina and not in the ocean
```

Part E — Reproducibility and AI use (1 point)

1. Your document renders from a clean session, with every package loaded explicitly. (*Check this last, after you have stopped editing.*)
2. End the document with an **AI use note**: which tools you used, what for, and at which stage — planning, coding, debugging, interpreting, or writing. Two or three sentences.

Remember the course policy: AI use is unrestricted and costs you nothing, but you must be able to explain every line you hand in. Disclosure is the only requirement.

Starting code

```
sessionInfo()
```

Rubric

Part	What is assessed	Points
A	Correct inspection of the object; clear, accurate answers	2
B	Correct transformation and areas; comparison explained	2
C	Both packages working; counts-vs-rates argument; class intervals compared and a defended choice; cartographic standards met	5
D	Point layer built with the right CRS and correctly overlaid	2
E	Renders from a clean session; AI use note present	1
Total		12

Writing counts for roughly 10% of the mark within each part: the prose should be in complete sentences and free of obvious errors. Every part asks for interpretation as well as code — a submission that is all code and no sentences cannot score above about 60%.

Getting help

- Office hours, or email me — with the code and the exact error message, not a screenshot of the whole console.
- The `sf` documentation is genuinely good: `vignette("sf1", package = "sf")`.
- Moraga Chapter 2 covers most of Parts A–C.
- If you are stuck for more than about thirty minutes on something that feels mechanical, that is a signal to ask rather than to persevere.